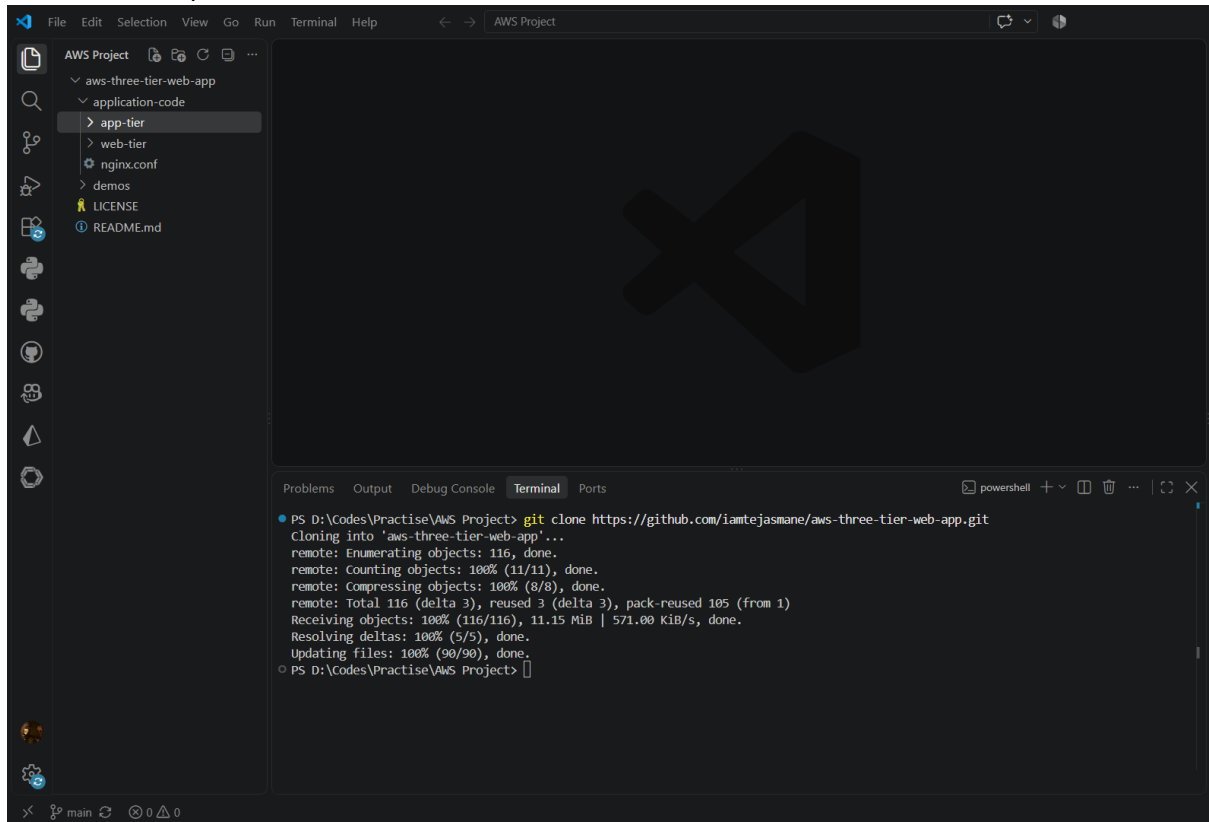


AWS Project

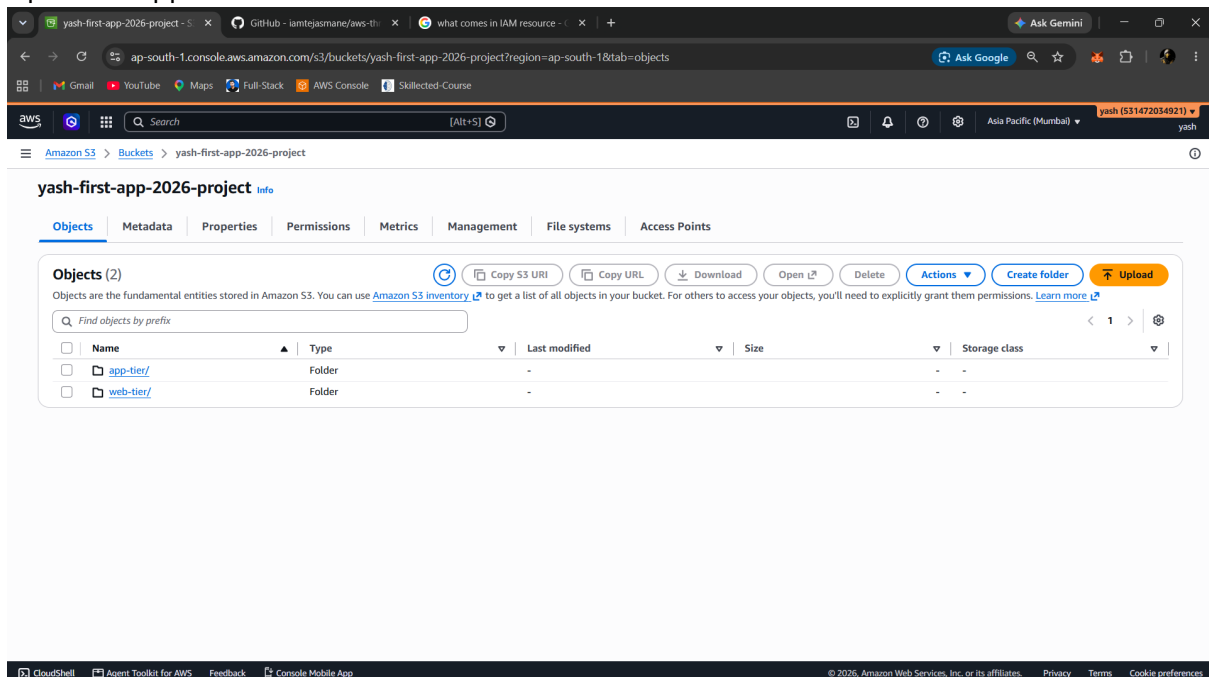
Cloned The Repo -



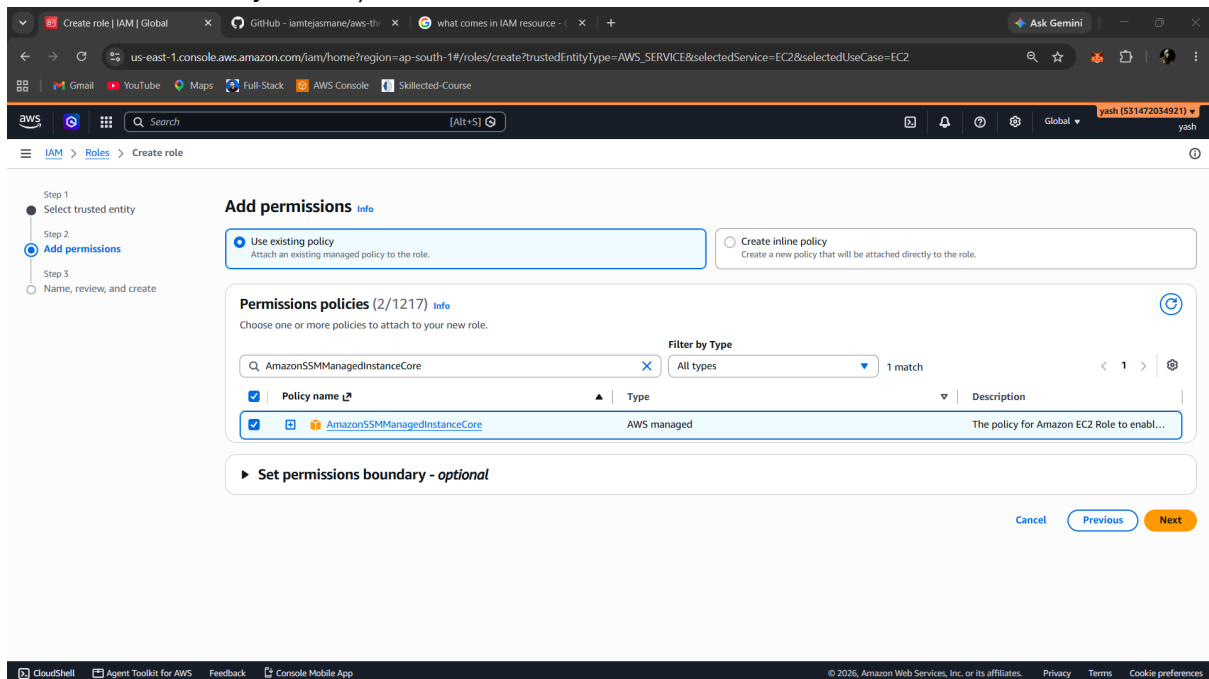
The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left displays the project structure: 'aws-three-tier-web-app' containing 'application-code', 'app-tier', 'web-tier', 'nginx.conf', 'demos', 'LICENSE', and 'README.md'. The main editor area is empty, showing a large dark blue 'X' logo. The Terminal panel at the bottom shows the command to clone the repository and its output.

```
PS D:\Codes\Practise\AWS Project> git clone https://github.com/iamtejasmene/aws-three-tier-web-app.git
Cloning into 'aws-three-tier-web-app'...
remote: Enumerating objects: 116, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 116 (delta 3), reused 3 (delta 3), pack-reused 105 (from 1)
Receiving objects: 100% (116/116), 11.15 MiB | 571.00 KiB/s, done.
Resolving deltas: 100% (5/5), done.
Updating files: 100% (90/90), done.
PS D:\Codes\Practise\AWS Project>
```

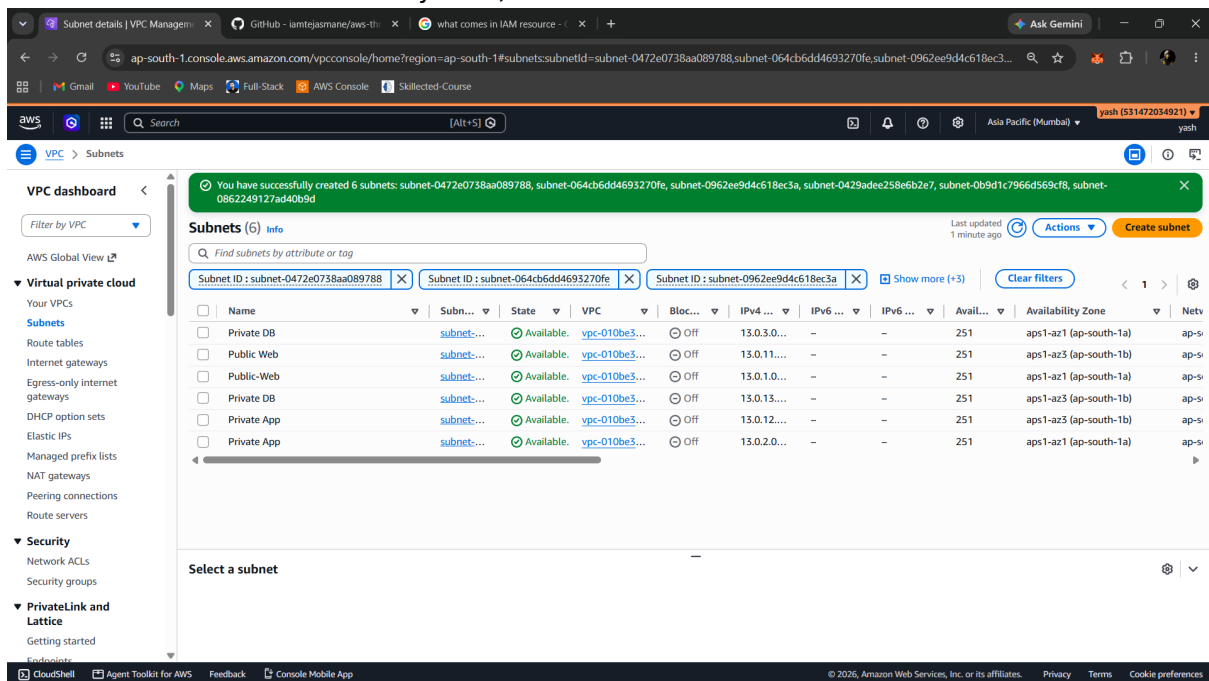
Uploaded app tier and web tier in s3 -



Created an IAM Role With Permissions (AmazonSSMManagedInstanceCore And AmazonS3ReadOnlyAccess) -



Created 6 subnets in 2 availability zone, 3 in 1a and 3 in 1b -



And then created igw and 2 Nat gateways and gave them In route table... -

2 subnets for public web rt,

2 subnets for private db RT,

1-1 subnet to private app az1 and az2

Nat gateway to – priv app 1a and 1b

Igw – public web rt

Created 2 Elastic Ips -

The screenshot shows the AWS Management Console for the 'ap-south-1' region. A green notification banner at the top states 'Elastic IP address allocated successfully. Elastic IP address 15.207.146.120'. Below this, the 'Elastic IP addresses (3)' page is displayed. It contains a table with the following data:

	Name	Allocated IPv4 address	Type	Allocation ID	Reverse DNS record	Associated instance ID
☐	First E-IP	13.232.87.24	Public IP	eipalloc-020d4c0944a7ab0ad	-	-
☐	-	15.206.216.77	Public IP	eipalloc-04d6a51414011684	-	-
☐	Second Elastic IP	15.207.146.120	Public IP	eipalloc-0111c78c533cb94e4	-	-

Below the table, there is a section 'Select an elastic IP address' with a link to 'View IP address usage and recommendations to release unused IPs with Public IP insights.' The left sidebar shows the navigation menu with categories like Virtual private cloud, Security, and PrivateLink and Lattice.

Created a 2 zonal NAT gateway and allocated the elastic ip 1 as well as public web 1a subnet and same for second -

The screenshot shows the 'Create NAT gateway' page in the AWS Management Console. The page is titled 'Create NAT gateway' and includes a description: 'A highly available, managed Network Address Translation (NAT) service that instances in private subnets can use to connect to services in other VPCs, on-premises networks, or the internet.' The 'NAT gateway settings' section is expanded, showing the following configuration:

- Name - optional:** Public-Web-AZ1
- Availability mode:** Zonal (selected). The 'Regional - new' option is also visible.
- Subnet:** subnet-0472e0738aa089788 (Public-Web-1a)
- Connectivity type:** Public (selected). The 'Private' option is also visible.
- Elastic IP allocation ID:** eipalloc-020d4c0944a7ab0ad (First E-IP). An 'Allocate Elastic IP' button is present.

Below the settings, there is an 'Additional settings' section. The bottom of the page shows the AWS footer with copyright information and links to Privacy, Terms, and Cookie preferences.

Created 3 route tables (2 private and 1 public) -

The screenshot shows the AWS Management Console for the 'Route tables' page. The left sidebar contains navigation links for VPC, Security, and PrivateLink and Lattice. The main content area shows a list of route tables. The 'Private DB Route Table' is selected, and its details are shown below the list.

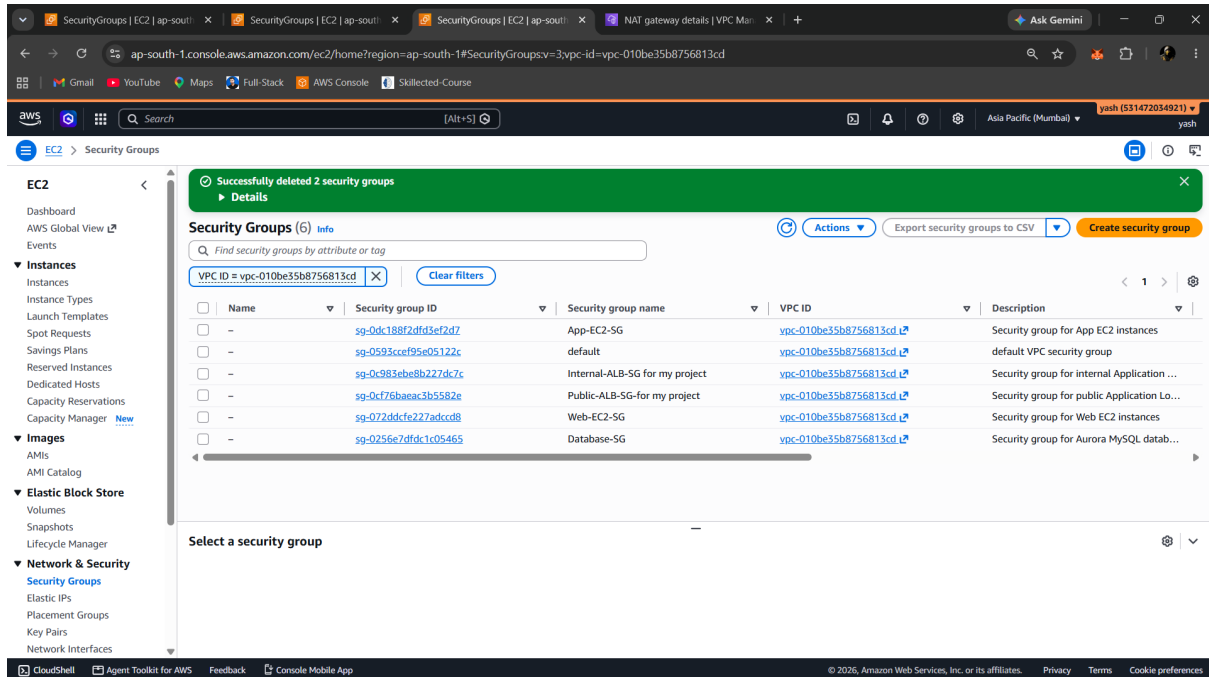
Name	Route table ID	Explicit subnet associations	Edge associations	Main	VPC	Owner ID
Public-Web-RT	rtb-0c6bee88c2cef5a14	2 subnets	-	Yes	vpc-010be35b8756813cd	531472034921
Private-App-AZ1	rtb-0415661fc88a0f152	subnet-064cb6dd469327...	-	No	vpc-010be35b8756813cd	531472034921
Private-App-AZ2	rtb-075abbe0b929fb676	-	-	No	vpc-010be35b8756813cd	531472034921
Private DB Route Table	rtb-014edf2edbecd7ad4	2 subnets	-	No	vpc-010be35b8756813cd	531472034921

Then created a DB Route table without IGW or NAT (without internet) -

The screenshot shows the AWS Management Console for the 'Route table details' page. The page displays the details of the 'Private DB Route Table' (rtb-059d50a58f45cdf26). The details include the route table ID, VPC, main status, owner ID, and explicit subnet associations.

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
Private-DB-1a	subnet-0962ee9d4c618ec3a	13.0.3.0/24	-
Private-DB-1b	subnet-0862249127ad40b9d	13.0.13.0/24	-

Created custom 5 security groups -



Their configurations –

Public-ALB-SG

HTTP 80 ← 0.0.0.0/0

HTTPS 443 ← 0.0.0.0/0

Web-EC2-SG

HTTP 80 ← Public-ALB-SG

Internal-ALB-SG

HTTP 80 ← Web-EC2-SG

App-EC2-SG

TCP 4000 ← Internal-ALB-SG

Database-SG

MySQL 3306 ← App-EC2-SG

Created RDS DB subnet group -

Subnet group details

Name
You won't be able to modify the name after your subnet group has been created.
myproject-SubnetGroup
Must contain from 1 to 255 characters. Alphanumeric characters, spaces, hyphens, underscores, and periods are allowed.

Description
DB subnet group for three-tier application

VPC
Choose a VPC identifier that corresponds to the subnets you want to use for your DB subnet group. You won't be able to choose a different VPC identifier after your subnet group has been created.
vpc-04 (vpc-010be35b8756813cd)
6 Subnets, 2 Availability Zones

Add subnets

Availability Zones
Choose the Availability Zones that include the subnets you want to add.
Choose an availability zone
ap-south-1a x ap-south-1b x

Subnets
Choose the subnets that you want to add. The list includes the subnets in the selected Availability Zones.
Select subnets
Private-DB-1b Subnet ID: subnet-0862249127ad40b9d CIDR: 13.0.13.0/24 Private-DB-1a Subnet ID: subnet-0962ee9d4c618ec3a CIDR: 13.0.3.0/24

Created an mysql DB –

Creating database database-1
Your database might take a few minutes to launch. You can use settings from database-1 to simplify configuration of suggested database add-ons while we finish creating your DB for you. [View connection details](#)

Databases (1)

Filter by databases

	DB identifier	Status	Role	Engine	Upgrad...	Region & AZ	Size	Recommendations
	database-1	Creating	Instance	MySQL Co...	SECOND	-	db.t4g.mi...	

Launched an instance (with existing security group and added the role to it)-

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required
first [Create new key pair](#)

▼ Network settings [Info](#)

VPC - required [Info](#)
vpc-010be35b8756813cd (vpc-04) 13.0.0.0/16 [Create new VPC](#)

Subnet [Info](#)
subnet-064cb6dd4693270fe [Create new subnet](#)
VPC: vpc-010be35b8756813cd Owner: 531472034921 Availability Zone: ap-south-1a (aps1-az1) Zone type: Availability Zone IP addresses available: 251 CIDR: 13.0.0.0/24

Auto-assign public IP [Info](#)
Disable

Firewall (security groups) [Info](#)
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.
☐ Create security group ☒ Select existing security group

Common security groups [Info](#)
Select security groups
App-EC2-SG sg-0dc188f2df3ef2d7 [Compare security group rules](#)
VPC: vpc-010be35b8756813cd
Security groups that you add or remove here will be added to or removed from all your network interfaces.

► Advanced network configuration

▼ Summary
[Number of instances](#) [Info](#)
1

Software image (AMI)
Amazon Linux 2023 AMI 2023.12...[read more](#)
ami-090d68841c2a28756

Virtual server type (instance type)
c7i-flex.large

Firewall (security group)
App-EC2-SG

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Configuring the instance –

- `sudo -su ec2-user`
- `cd ~`
- `ping 8.8.8.8` (check the internet)
- `sudo dnf install mariadb105 -y`
- `mysql -h database-1.cjcimuko8hn1.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p`
- `SHOW DATABASES;`
- `USE webappdb;`
- `CREATE TABLE IF NOT EXISTS transactions (`

`id INT NOT NULL AUTO_INCREMENT,`

`amount DECIMAL(10,2),`

`description VARCHAR(100),`

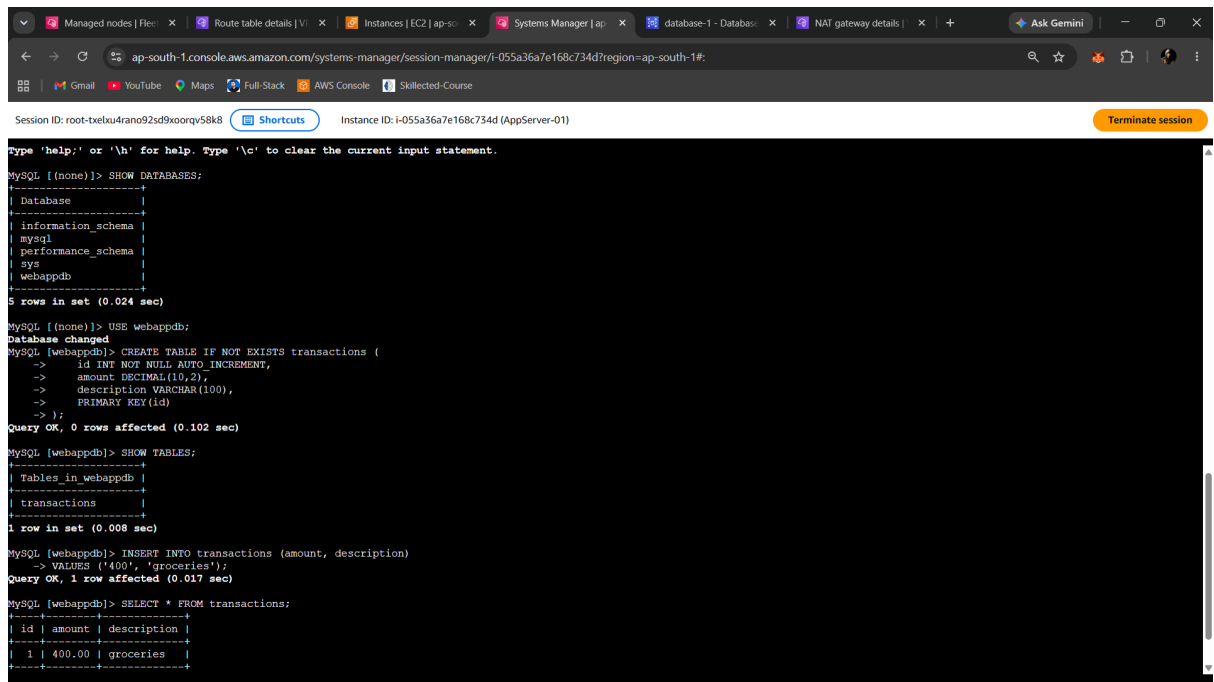
`PRIMARY KEY(id)`

`);`

- `SHOW TABLES;`
- `INSERT INTO transactions (amount, description)`

`VALUES ('400', 'groceries');`

- `INSERT INTO transactions (amount, description)`
`VALUES ('400', 'groceries');`
- `Exit`



```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| webappdb |
+-----+
5 rows in set (0.024 sec)

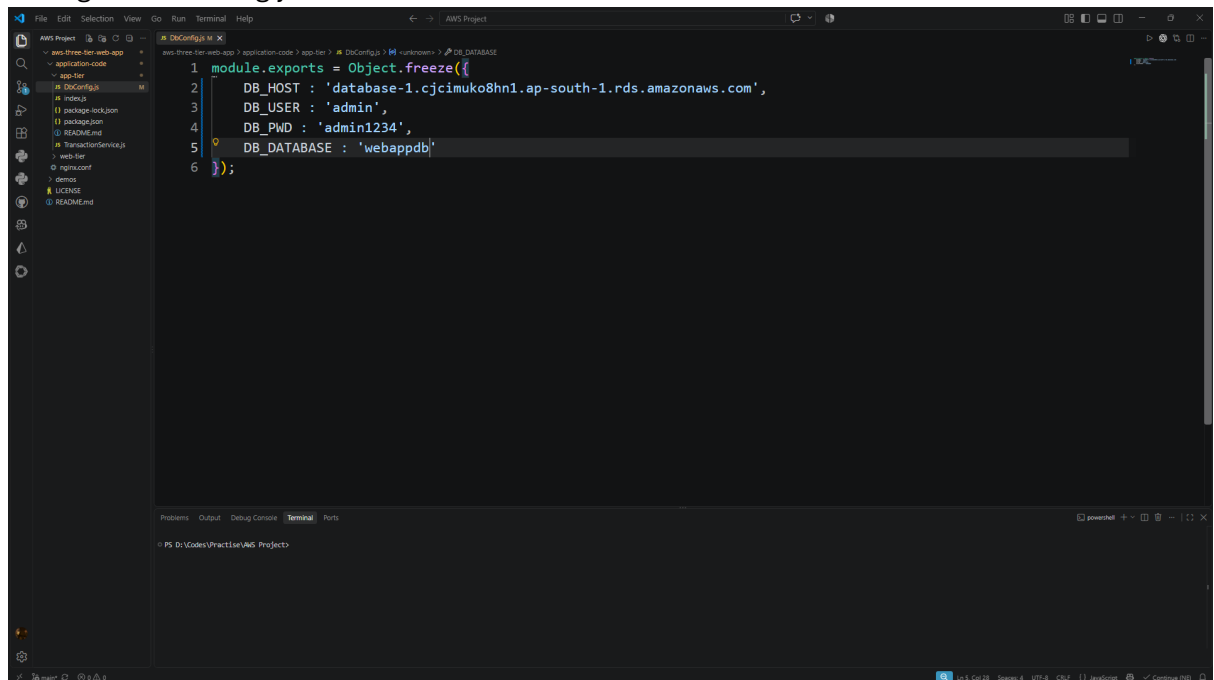
MySQL [(none)]> USE webappdb;
Database changed
MySQL [webappdb]> CREATE TABLE IF NOT EXISTS transactions (
  ->   id INT NOT NULL AUTO_INCREMENT,
  ->   amount DECIMAL(10,2),
  ->   description VARCHAR(100),
  ->   PRIMARY KEY(id)
  -> );
Query OK, 0 rows affected (0.102 sec)

MySQL [webappdb]> SHOW TABLES;
+-----+
| Tables_in_webappdb |
+-----+
| transactions |
+-----+
1 row in set (0.008 sec)

MySQL [webappdb]> INSERT INTO transactions (amount, description)
  -> VALUES ('400', 'groceries');
Query OK, 1 row affected (0.017 sec)

MySQL [webappdb]> SELECT * FROM transactions;
+----+-----+-----+
| id | amount | description |
+----+-----+-----+
| 1 | 400.00 | groceries |
+----+-----+-----+
```

Configured Dbconfig.js -



```
1 module.exports = Object.freeze({
2   DB_HOST : 'database-1.cjcimuko8hn1.ap-south-1.rds.amazonaws.com',
3   DB_USER : 'admin',
4   DB_PWD : 'admin1234',
5   DB_DATABASE : 'webappdb'
6 });
```

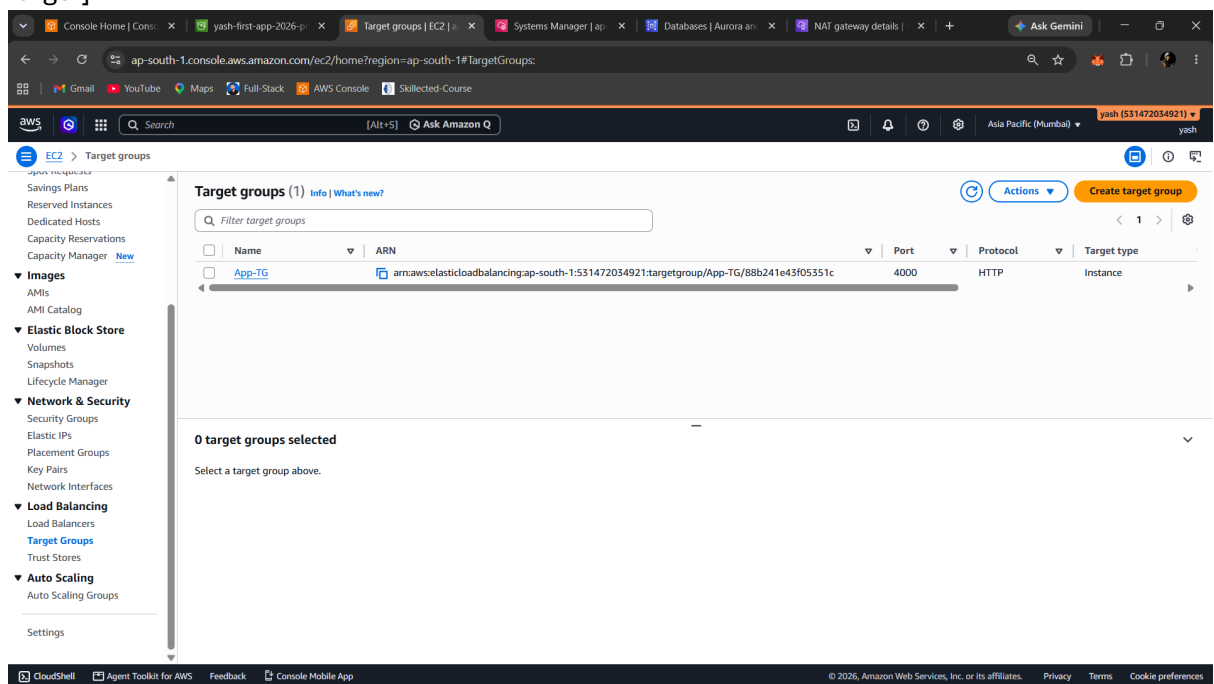
Deleted the old app/dbconfig.js from s3 and uploaded the new...

Now installing node –

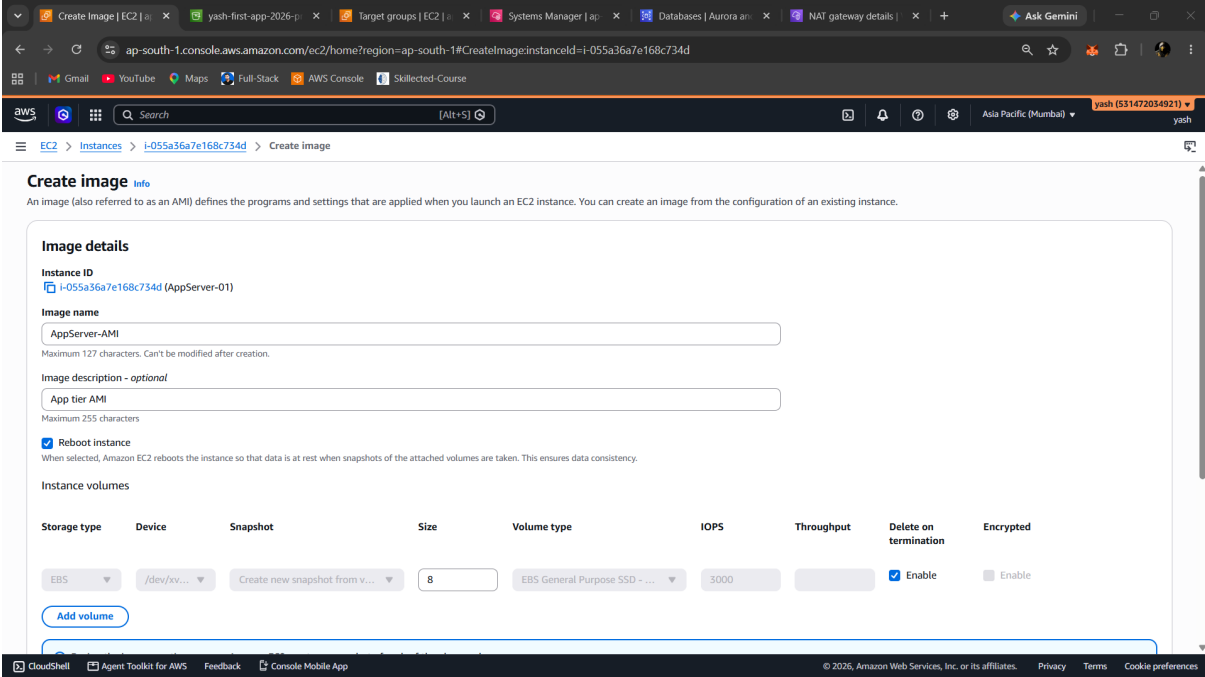
- `cd ~`
- `curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.38.0/install.sh | bash`
- `source ~/.bashrc`
- `nvm install 16`
- `nvm use 16`
- `node -v` (verify it)
- `npm -v` (verify it)

- `npm install -g pm2`
- `cd ~`
- `aws s3 cp s3://YOUR-BUCKET-NAME/app-tier/ app-tier --recursive`
- `ls ~/app-tier` (check the files)
- app dependencies- `cd ~/app-tier` `npm install`
- `pm2 start index.js` (starting node app)
- `pm2 startup` (it will print the path) (`sudo env PATH=$PATH:/home/ec2-user/.nvm/versions/node/v16.20.2/bin /home/ec2-user/.nvm/versions/node/v16.20.2/lib/node_modules/pm2/bin/pm2 startup systemd -u ec2-user --hp /home/ec2-user`)
- Run that command
- `Pm2 save`
- `curl http://localhost:4000/health` (output "This is the health check")
- `curl http://localhost:4000/transaction` (output - id: 1 amount: 400.00 description: groceries)

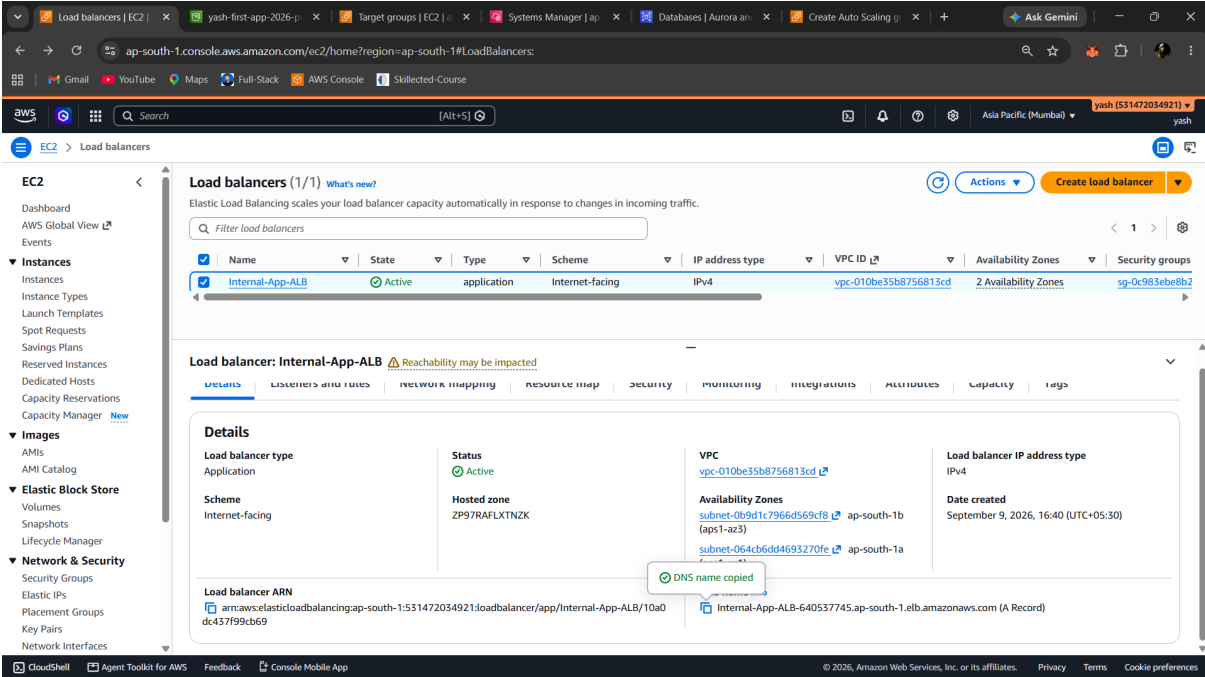
Created an target group for app – (path - /health, port - 4000) [but without selecting the target] -



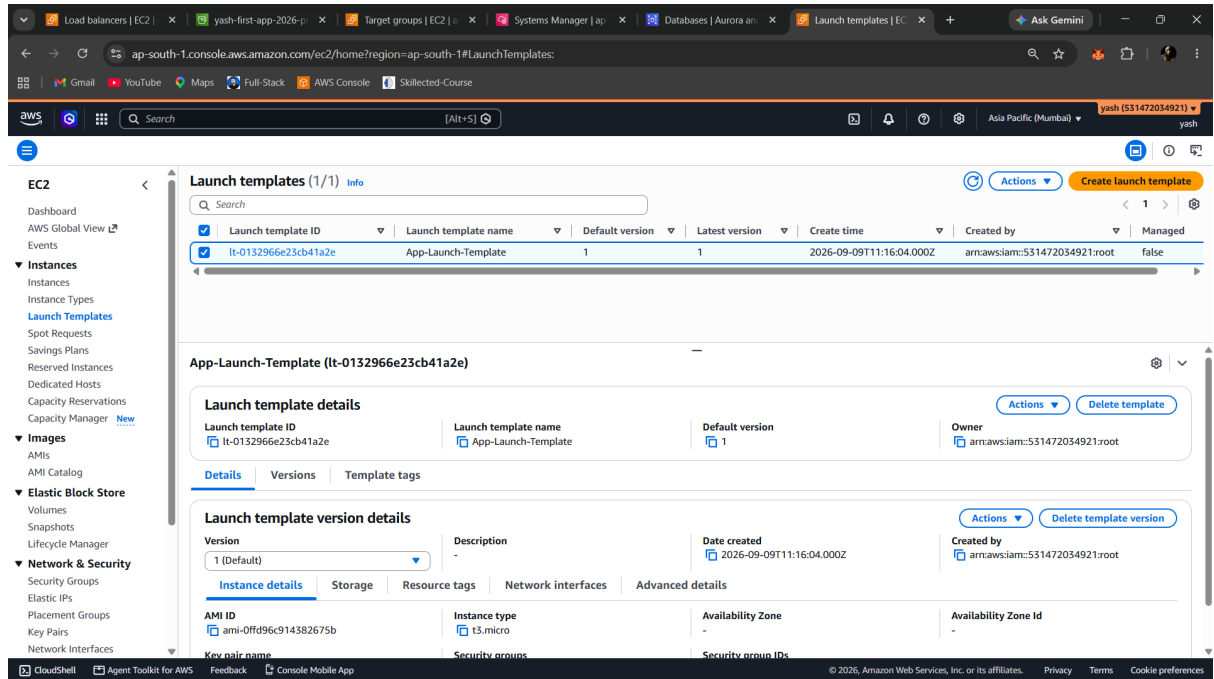
Created an image of app server instance -



Created a Application Load Balancer for App (with Internal Schema) –



Created Launch Template -



Then Created an AutoScaling Group –

[Subnet - Private-App-AZ1 Private-App-AZ2,

Attach to an existing load balancer – App-TG

Group size - Desired capacity: 2

Minimum capacity: 2

Maximum capacity: 2

You should eventually have (in EC2- AppServer-01 + 2 new App-ASG instances) -

The screenshot displays the AWS Management Console interface, specifically the Auto Scaling groups and EC2 instances sections.

Auto Scaling groups (1/1)

Search your Auto Scaling groups

Name	Status	Launch template/configuration	Instances	Instance health	Desired capacity	Min	Max	Availability Zones
App-ASG	Updating capacity	App-Launch-Template	Version Default	0	-	2	2	2 Availability Zones

Auto Scaling group: App-ASG

Details | Integrations | Automatic scaling | Instance management | Instance refresh | Activity | Monitoring | Tags

App-ASG Capacity overview

arn:aws:autoscaling:ap-south-1:531472034921:autoScalingGroup:53072553-6757-4fa2-9001-fd184a333de3:autoScalingGroupName/App-ASG

Desired capacity: 2 | Scaling limits: 2 - 2 | Desired capacity type: Units (number of instances) | Status: Updating capacity

Date created: Wed Sep 09 2026 16:49:54 GMT+0530 (India Standard Time)

Launch template

Launch template	AMI ID	Instance type	Owner

Instances (3)

Filter Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
	i-0455b211a5225c636	Running	t3.micro	Initializing	ap-south-1b	-	-
AppServer-01	i-055a36a7e168c734d	Running	t3.micro	3/3 checks passed	ap-south-1a	-	-
	i-0a97bcb24c1a5ed0a	Running	t3.micro	Initializing	ap-south-1a	-	-

All instances

Instance alarm states

In alarm: None

Service health

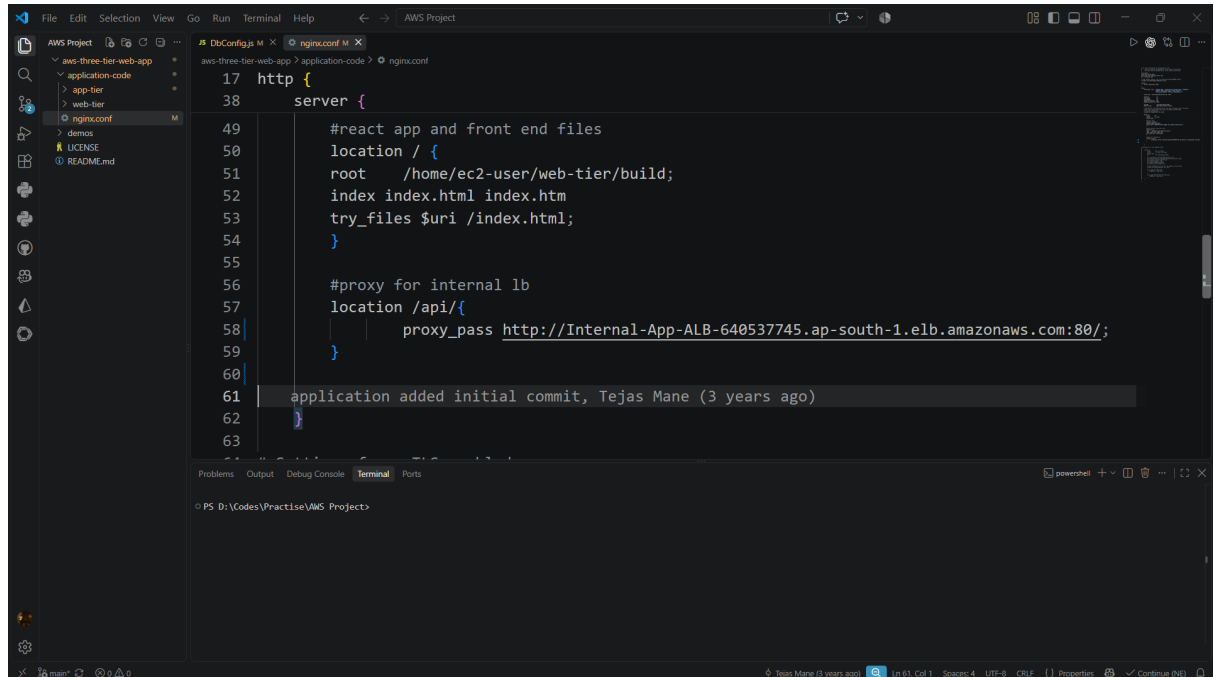
EC2 service status: Operating normally

Health events affecting EC2 instances

Open events: None

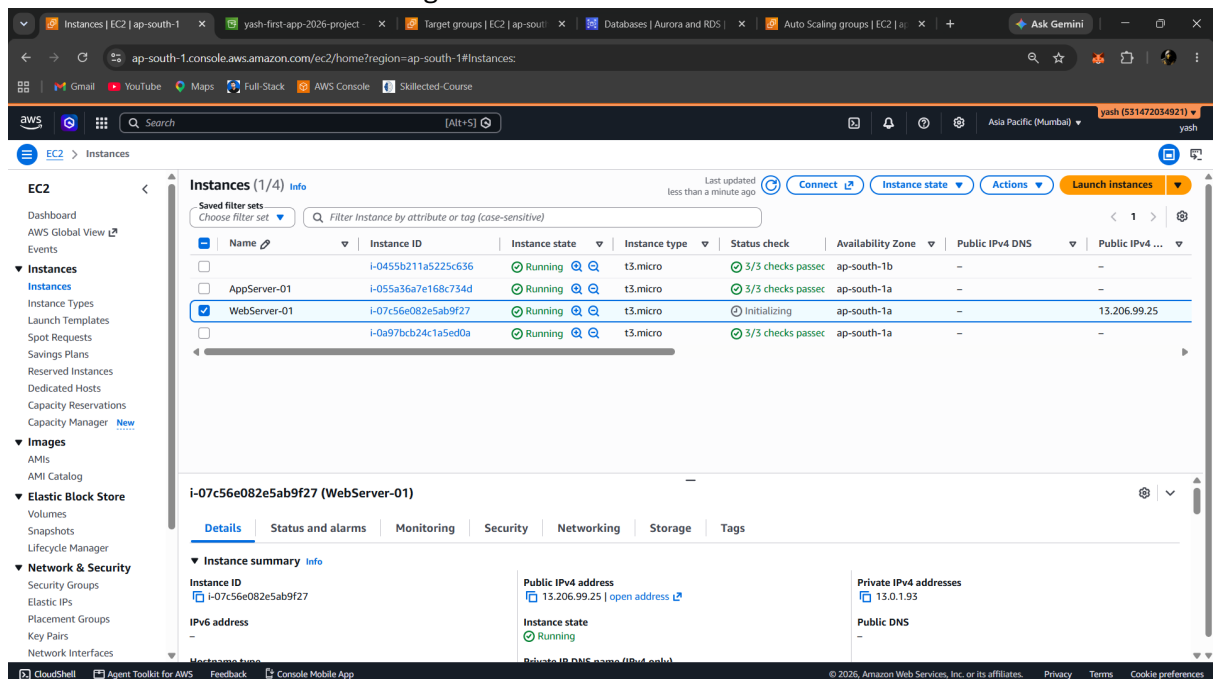
Upcoming events: None

Edit nginx.conf and uploaded this nginx.conf file onto the s3 bucket (root folder)-



```
17 http {
38     server {
49         #react app and front end files
50         location / {
51             root    /home/ec2-user/web-tier/build;
52             index index.html index.htm
53             try_files $uri /index.html;
54         }
55
56         #proxy for internal lb
57         location /api/{
58             proxy_pass http://Internal-App-ALB-640537745.ap-south-1.elb.amazonaws.com:80;
59         }
60
61         application added initial commit, Tejas Mane (3 years ago)
62     }
63 }
```

Created another new instance now (with auto assign ip enable, everything else is same) and connected as SSM session manager -

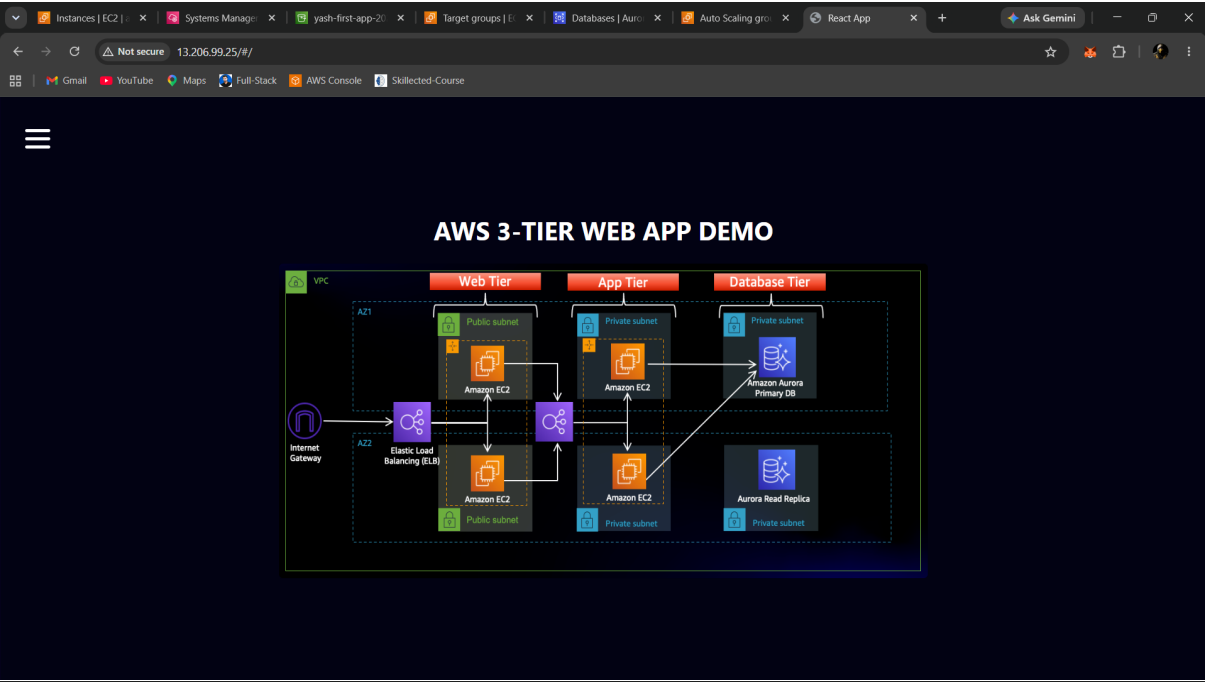


Commands –

- `sudo -su ec2-user`
- `cd ~`
- `curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.38.0/install.sh | bash`
- `source ~/.bashrc`
- `nvm install 16`
- `nvm use 16`
- `aws s3 cp s3://yash-first-app-2026-project/web-tier/ web-tier --recursive`

- `cd ~/web-tier`
- `npm install`
- `npm run build` (example output - `~/web-tier/build`)
- `sudo dnf install nginx -y` (for verification - `nginx -v`)
- `cd /etc/nginx` (verify using `ls`, output – `conf.d`)
- `sudo rm nginx.conf` (removing the default config)
- `sudo aws s3 cp s3://yash-first-app-2026-project/nginx.conf .`
(installing the original config) (`ls -l nginx.conf` – check it)
- `sudo nginx -t` (sample output)-
nginx: the configuration file `/etc/nginx/nginx.conf` syntax is ok
nginx: configuration file `/etc/nginx/nginx.conf` test is successful
- `chmod -R 755 /home/ec2-user`
- `sudo systemctl enable nginx`
- `sudo systemctl restart nginx`

Now copy the Public IPV4 of Web server and search it in the browser (you should see the React Website) –



Instances | EC2 | x Systems Manage x yash-first-app-20 x Target groups | E x Databases | Auto x Auto Scaling gro x React App x + Ask Gemini

Not secure 13.206.99.25/#/db

Gmail YouTube Maps Full-Stack AWS Console Skilleded-Course

AURORA DATABASE DEMO PAGE

DEL

ID	AMOUNT	DESC
ADD		

HOME

DB DEMO

Created another Application Load Balancer (forward to web-tg) -

Availability Zones and subnets [Info](#)

Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

☒ **ap-south-1a (aps1-az1)**

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-0472e0738aa089788 (Public-Web-1a)
13.0.1.0/24
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (-/)

☒ **ap-south-1b (aps1-az3)**

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently. Selected subnets must have Network ACL inbound and outbound ALLOW rules, and internet gateway (IGW) routes configured to allow traffic.

subnet-0429adee258e6b2e7 (Public-Web-1b)
13.0.11.0/24
Network ACL ALLOW rules: Inbound (1) Outbound (1) | Route table: IGW (-/)

Security groups [Info](#)

A security group is a set of firewall rules that control the traffic to your load balancer. Listed security groups are in the same VPC as you selected above. Selected security groups must include at least 1 inbound rule and 1 outbound rule to allow traffic to your load balancer.

Security groups

Select up to 5 security groups

Public-ALB-SG-for my project (sg-0cf76baec3b5582e) [X](#)
Inbound rule (2) Outbound rule (1)

Listeners and routing [Info](#)

Created AMI of Web Server -

Create image [Info](#)

An image (also referred to as an AMI) defines the programs and settings that are applied when you launch an EC2 instance. You can create an image from the configuration of an existing instance.

Image details

Instance ID

i-07c56e082e5ab9f27 (WebServer-01)

Image name

WebServer-AMI
Maximum 127 characters. Can't be modified after creation.

Image description - optional

WebServer-AMI
Maximum 255 characters

☒ **Reboot instance**

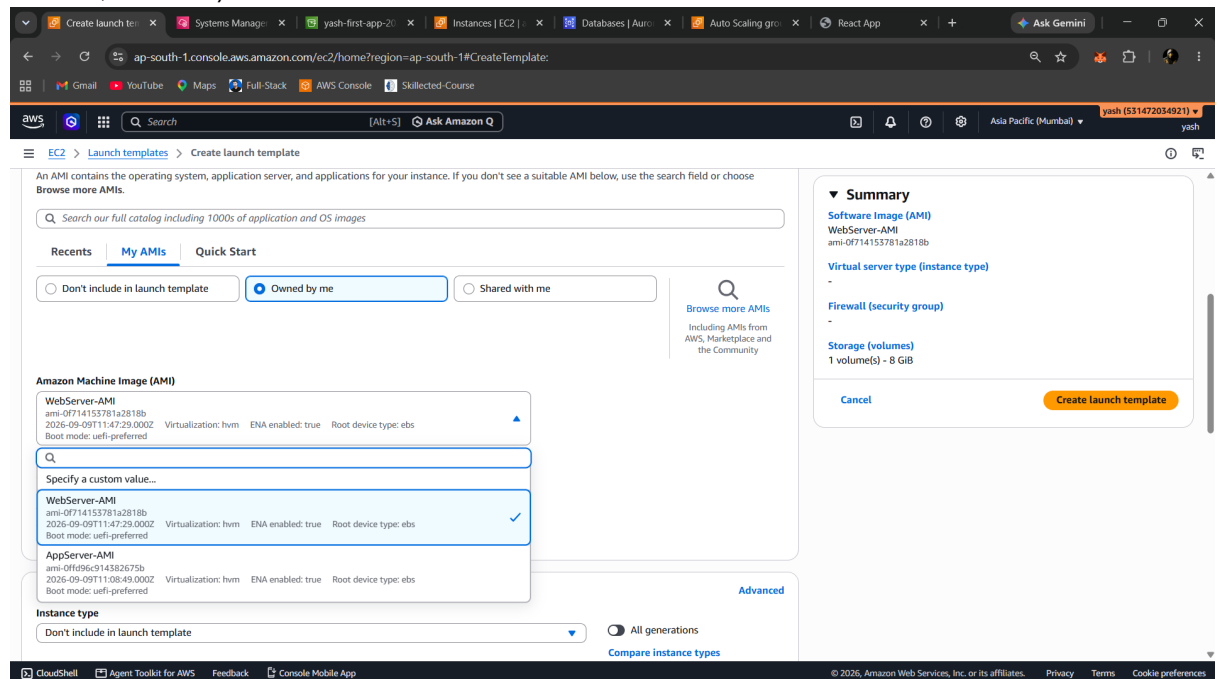
When selected, Amazon EC2 reboots the instance so that data is at rest when snapshots of the attached volumes are taken. This ensures data consistency.

Instance volumes

Storage type	Device	Snapshot	Size	Volume type	IOPS	Throughput	Delete on termination	Encrypted
EBS	/dev/xv...	Create new snapshot from v...	8	EBS General Purpose SSD - ...	3000		☒ Enable	☐ Enable

[Add volume](#)

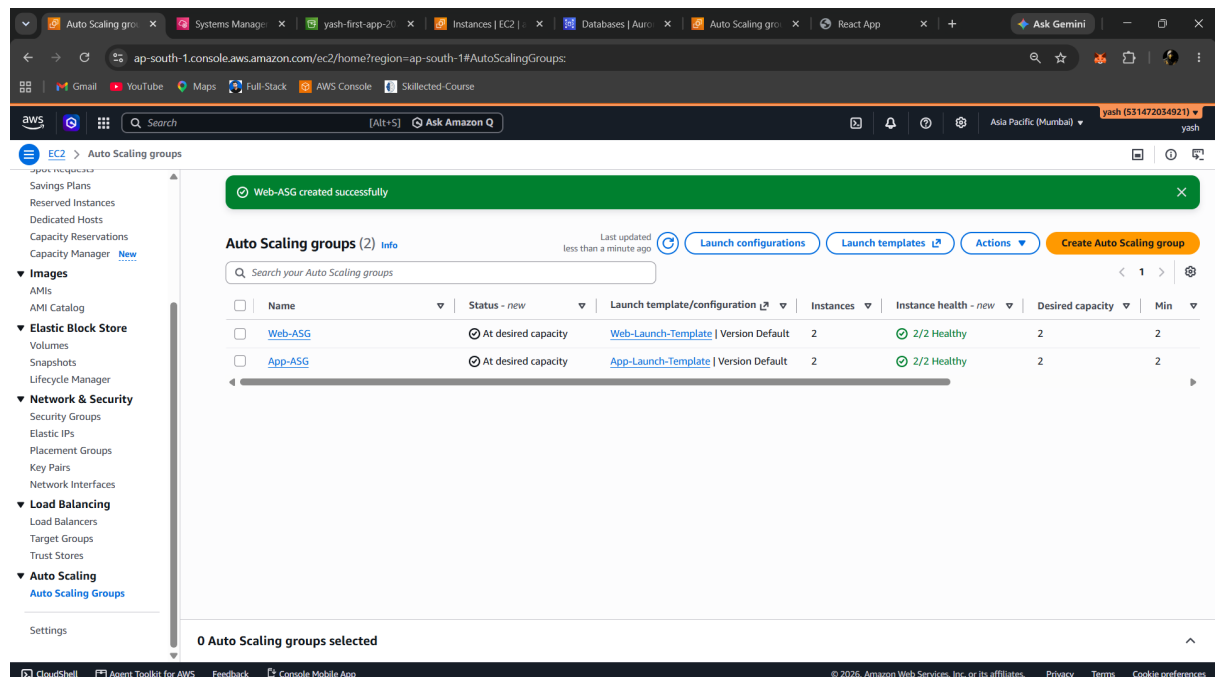
Launch the template of web server (no key, no subnet, no zone, security group – web ec2 SG, and IAM)-



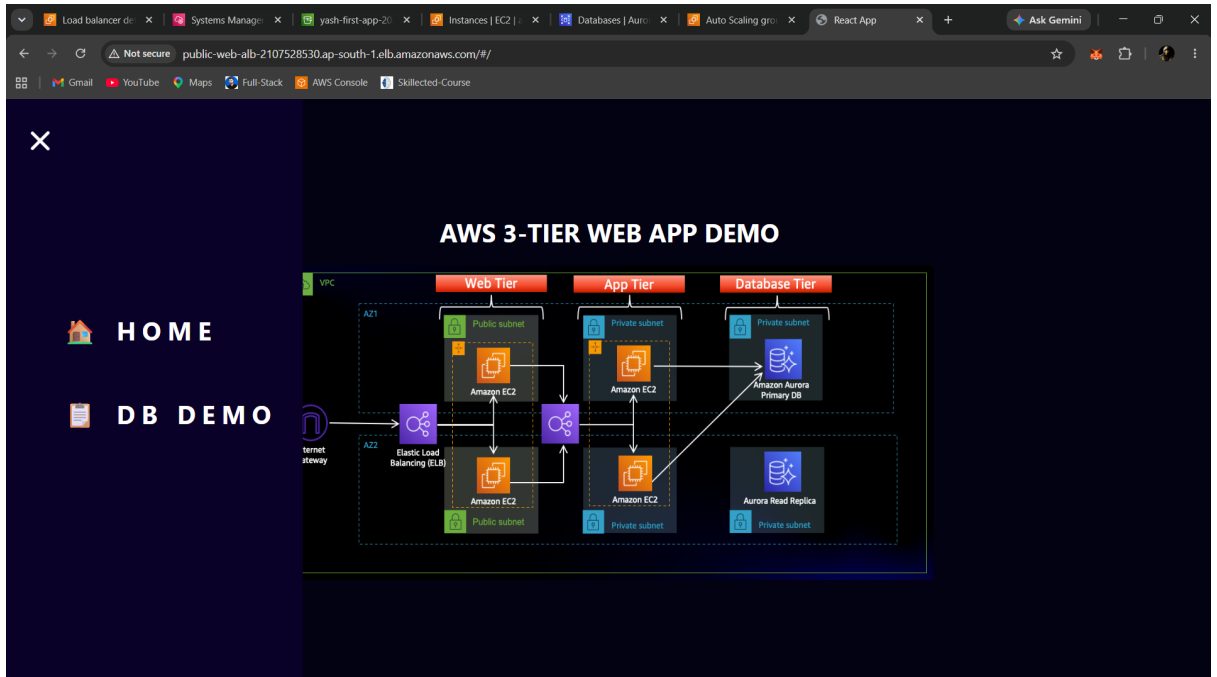
Auto scaling groups (availability zones – publis 1a and 1b, existing LB -web-TG, Desired capacity: 2

Minimum capacity: 2

Maximum capacity: 2)-

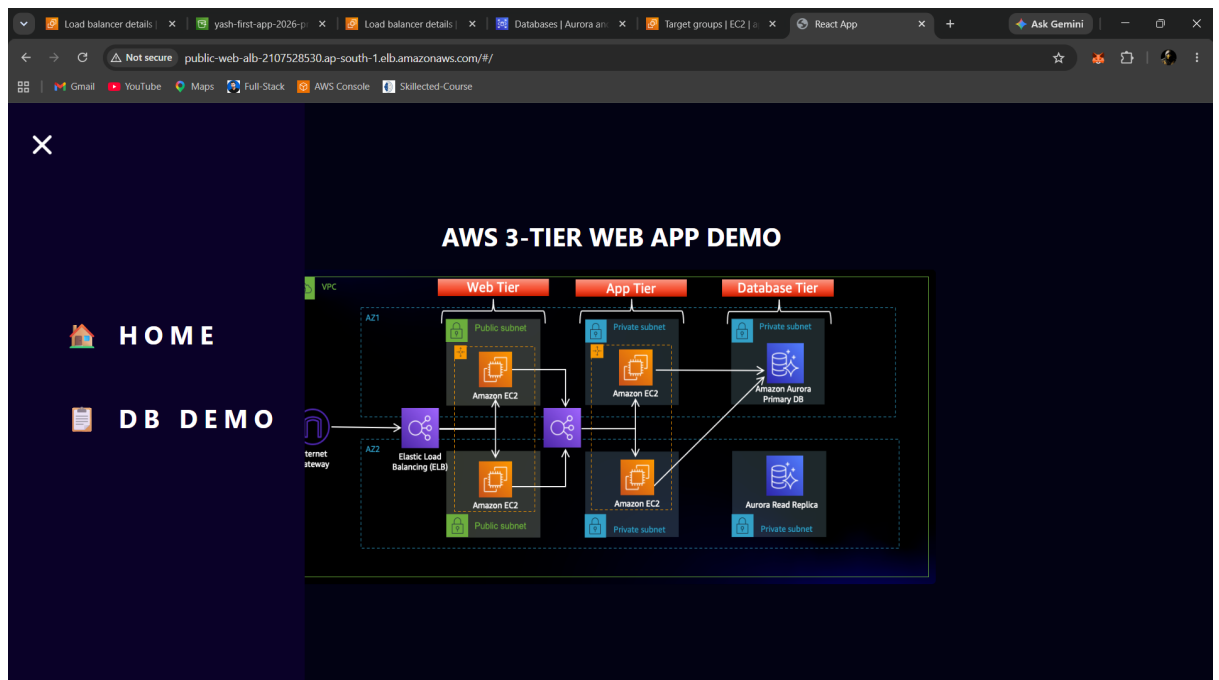


Checked weather the LB is working -

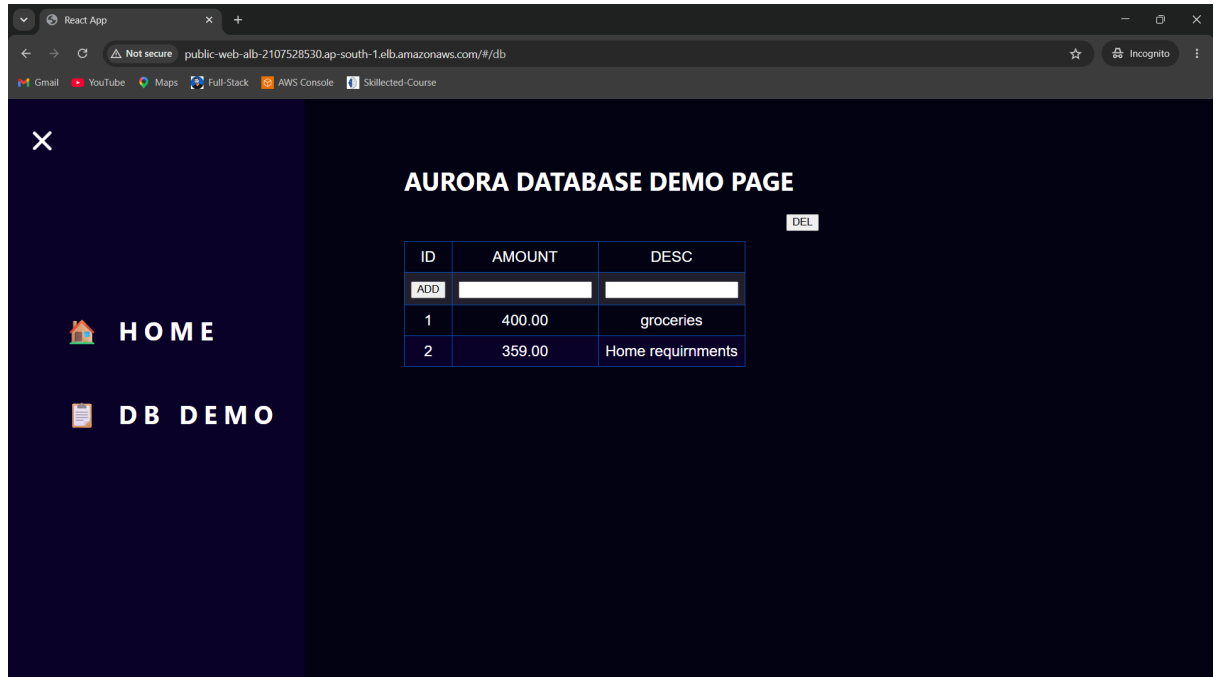


❖ Testing –

1. Web TG home -



2. Databse connected -



AWS Three-Tier Web Application Deployment

1. Project Overview

The project implements a three-tier web application on AWS consisting of:

Internet



Public Application Load Balancer



Web Tier



Internal Application Load Balancer



Application Tier



RDS MySQL Database

The deployment uses:

- Amazon VPC
- Public and private subnets

- Internet Gateway
- NAT Gateways
- Route Tables
- Security Groups
- Amazon S3
- IAM
- EC2
- Application Load Balancers
- Auto Scaling Groups
- Amazon RDS MySQL
- Nginx
- React
- Node.js

PART 2 — VPC

VPC

6 Subnets:

Public Web Subnets

- Public-Web-AZ1
- Public-Web-AZ2

These subnets are used by the public-facing Web tier and Public Application Load Balancer.

Private Application Subnets

- Private-App-AZ1
- Private-App-AZ2

These subnets contain the Application tier.

Private Database Subnets

- Private-DB-AZ1
- Private-DB-AZ2

These subnets are used by Amazon RDS.

PART 3 — INTERNET GATEWAY

Internet Gateway

PART 4 — NAT GATEWAYS

Two NAT Gateways were created for high availability.

- NAT-AZ1
- NAT-AZ2

They were placed in:

- Public-Web-AZ1
- Public-Web-AZ2

Each NAT Gateway was associated with an Elastic IP.

PART 5 — ROUTE TABLES

Public Web Route Table

Public-Web-RT

Routes:

13.0.0.0/16 → local

0.0.0.0/0 → Internet Gateway

Associated with:

- Public-Web-AZ1
 - Public-Web-AZ2
-

Private App Route Table — AZ1

Private-App-AZ1

Routes:

13.0.0.0/16 → local

0.0.0.0/0 → NAT-AZ1

Associated with:

- Private-App-AZ1
-

Private App Route Table — AZ2

Private-App-AZ2

Routes:

13.0.0.0/16 → local

0.0.0.0/0 → NAT-AZ2

Associated with:

- **Private-App-AZ2**
-

Private Database Route Table

Private-DB-RT

Route:

13.0.0.0/16 → local

Associated with:

- **Private-DB-AZ1**
 - **Private-DB-AZ2**
-

PART 6 — SECURITY GROUPS

Five security groups were configured.

1. Public-ALB-SG

Inbound:

HTTP

Port: 80

Source: 0.0.0.0/0

2. Web-EC2-SG

Inbound:

HTTP

Port: 80

Source: Public-ALB-SG

3. Internal-ALB-SG

Inbound:

HTTP

Port: 80

Source: Web-EC2-SG

4. App-EC2-SG

Inbound:

TCP

Port: 4000

Source: Internal-ALB-SG

5. Database-SG

Inbound:

MySQL/Aurora

Port: 3306

Source: App-EC2-SG

PART 7 — IAM ROLE

Created an EC2 IAM role:

ThreeTierEC2Role-for-my-project

Policies attached:

AmazonSSMManagedInstanceCore

AmazonS3ReadOnlyAccess

The role allows:

- **Session Manager access**
 - **EC2 instances to read files from S3**
-

PART 8 — S3 BUCKET

Created S3 bucket:

yash-first-app-2026-project

The bucket was structured as:

yash-first-app-2026-project/

```
|  
|  
|— app-tier/  
|  
|— web-tier/  
|  
|— nginx.conf
```

The application code was uploaded to S3 so EC2 instances could retrieve it using the AWS CLI.

PART 9 — RDS MYSQL DATABASE

Regular Amazon RDS MySQL database was created.

Configuration

Database name:

webappdb

Master username:

admin

Public access:

No

The database was placed in the private DB subnets.

PART 10 — RDS SECURITY

RDS uses:

Database-SG

Inbound MySQL traffic:

TCP 3306

Source: App-EC2-SG

Therefore, the database cannot be directly accessed from the Internet.

PART 11 — DATABASE TABLE

Connected to RDS from the App EC2 instance.

Installed MySQL-compatible CLI:

```
sudo dnf install mariadb105 -y
```

Connected using:

```
mysql -h database-1.cjcmuko8hn1.ap-south-1.rds.amazonaws.com -u admin -p
```

Selected the database:

```
USE webappdb;
```

Created the table:

```
CREATE TABLE IF NOT EXISTS transactions(  
  id INT NOT NULL AUTO_INCREMENT,  
  amount DECIMAL(10,2),  
  description VARCHAR(100),  
  PRIMARY KEY(id)  
);
```

Inserted test data:

```
INSERT INTO transactions  
(amount,description)  
VALUES  
('400','groceries');
```

Verified:

```
SELECT * FROM transactions;
```

Sample output

```
+----+-----+-----+  
| id | amount | description |  
+----+-----+-----+  
| 1 | 400.00 | groceries  |  
+----+-----+-----+
```

PART 12 — APP EC2 INSTANCE

Created:

AppServer-01

Configuration

Instance type:

t3.micro

AMI:

Amazon Linux 2023

Availability Zone:

ap-south-1a

Private IP:

13.0.2.107

Public IP:

None

Security Group:

App-EC2-SG

IAM Profile:

ThreeTierEC2Role-for-my-project

The instance was accessed using:

EC2 → Connect → Session Manager

No SSH key was required.

PART 13 — APP TIER CODE

Downloaded the application code from S3:

aws s3 cp s3://yash-first-app-2026-project/app-tier/ app-tier --recursive

Entered the application directory:

cd ~/app-tier

PART 14 — NODE.JS

Installed Node.js using NVM.

curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.38.0/install.sh | bash

Loaded NVM:

source ~/.bashrc

Installed Node.js:

nvm install 16

Selected Node.js:

nvm use 16

PART 15 — DATABASE CONFIGURATION

Configured:

```
module.exports = Object.freeze({  
  DB_HOST : 'database-1.cjcmuko8hn1.ap-south-1.rds.amazonaws.com',  
  DB_USER : 'admin',  
  DB_PWD : 'RDS_PASSWORD',  
  DB_DATABASE : 'webappdb'  
});
```

PART 16 — MYSQL2 FIX

The original application used:

```
const mysql = require('mysql');
```

Because the RDS MySQL 8.4 authentication configuration caused compatibility problems with the older mysql package, the dependency was changed to mysql2.

Removed:

```
npm uninstall mysql
```

Installed:

```
npm install mysql2
```

Changed TransactionService.js to:

```
const mysql = require('mysql2');
```

PART 17 — INSTALL APP DEPENDENCIES

Inside:

```
~/app-tier
```

ran:

```
npm install
```

PART 18 — PM2

Installed PM2:

npm install pm2 -g

Started the application:

pm2 start index.js

Saved the PM2 process:

pm2 save

PART 19 — APP TIER TEST

Health test:

curl http://localhost:4000/health

Output

"This is the health check"

Transaction test:

curl http://localhost:4000/transaction

Output

{"result":[{"id":1,"amount":"400.00","description":"groceries"}]}

This confirmed that:

Node.js → RDS MySQL

was working.

PART 20 — APP TARGET GROUP

Created:

App-TG

Configuration

Target type:

Instances

Protocol:

HTTP

Port:

4000

VPC:

vpc-010be35b8756813cd

Health check

Protocol:

HTTP

Path:

/health

No target was manually registered.

PART 21 — APP AMI

Created an AMI from:

AppServer-01

Configuration

AMI Name:

AppServer-AMI

Description:

App tier AMI

The AMI was later used by the App Launch Template.

PART 22 — INTERNAL APPLICATION LOAD BALANCER

Created:

Internal-App-ALB

Configuration

Type:

Application Load Balancer

Scheme:

Internal

IP address type:

IPv4

VPC:

vpc-010be35b8756813cd

Subnets:

Private-App-AZ1

Private-App-AZ2

Security group:

Internal-ALB-SG

Listener:

HTTP : 80

Default action:

Forward to App-TG

These are the documented configuration values for the internal ALB.

PART 23 — APP LAUNCH TEMPLATE

Created:

App-Launch-Template

Configuration

AMI:

AppServer-AMI

Instance type:

t3.micro

Key pair:

Don't include

Security Group:

App-EC2-SG

Subnet:

Not selected

IAM Profile:

ThreeTierEC2Role-for-my-project

The subnet is selected later by the Auto Scaling Group.

PART 24 — APP AUTO SCALING GROUP

Created:

App-ASG

Launch template:

App-Launch-Template

VPC:

vpc-010be35b8756813cd

Subnets:

Private-App-AZ1

Private-App-AZ2

Load balancer target group:

App-TG

Capacity

Desired: 2

Minimum: 2

Maximum: 2

The target group was then verified to contain healthy application instances.

PART 25 — WEB TIER NGINX CONFIGURATION

The Nginx configuration was modified to forward API requests to the Internal ALB.

```
location /api/{  
    proxy_pass http://INTERNAL-ALB-DNS:80;  
}
```

The React application is served from:

/home/ec2-user/web-tier/build;

The index configuration was:

index index.html index.htm;

The /health endpoint was configured as:

```
location /health {  
    default_type text/html;  
    return 200 "<!DOCTYPE html><p>Web Tier Health Check</p>\n";  
}
```

The completed configuration was uploaded to S3 as:

nginx.conf

at the bucket root.

PART 26 — WEB EC2 INSTANCE

Created:

WebServer-01

Configuration

AMI:

Amazon Linux 2023

Architecture:

64-bit x86

Instance type:

t3.micro

Key pair:

Don't include

VPC:

vpc-010be35b8756813cd

Subnet:

Public-Web-AZ1

Public IP:

Enabled

Security Group:

Web-EC2-SG

IAM Profile:

ThreeTierEC2Role-for-my-project

PART 27 — WEB CODE

Connected to WebServer-01 using:

Session Manager

Installed Node.js 16 using NVM.

Downloaded the React application:

aws s3 cp s3://yash-first-app-2026-project/web-tier/ web-tier --recursive

Entered the directory:

```
cd ~/web-tier
```

Installed dependencies:

```
npm install
```

Built the React application:

```
npm run build
```

The production build was created at:

```
/home/ec2-user/web-tier/build
```

PART 28 — NGINX INSTALLATION

Installed Nginx:

```
sudo dnf install nginx -y
```

Verified:

```
nginx -v
```

Removed the default configuration:

```
cd /etc/nginx
```

```
sudo rm nginx.conf
```

Downloaded the custom configuration:

```
sudo aws s3 cp s3://yash-first-app-2026-project/nginx.conf .
```

Tested:

```
sudo nginx -t
```

Sample output

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
```

```
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Started Nginx:

```
sudo systemctl enable nginx
```

```
sudo systemctl restart nginx
```

PART 29 — WEB TARGET GROUP

Created:

Web-TG

Configuration

Target type:

Instances

Protocol:

HTTP

Port:

80

VPC:

vpc-010be35b8756813cd

Health check

Protocol:

HTTP

Path:

/health

No target was manually registered.

The target group was later populated automatically by the Web ASG.

PART 30 — PUBLIC APPLICATION LOAD BALANCER

Created:

Public-Web-ALB

Configuration

Type:

Application Load Balancer

Scheme:

Internet-facing

IP address type:

IPv4

VPC:

vpc-010be35b8756813cd

Subnets:

Public-Web-AZ1

Public-Web-AZ2

Security group:

Public-ALB-SG

Listener:

HTTP : 80

Default action:

Forward to Web-TG

PART 31 — WEB AMI

Created an AMI from the fully configured WebServer-01.

Initial AMI:

WebServer-AMI

After the Internal ALB was recreated correctly, the Nginx configuration was updated and a second AMI was created:

WebServer-AMI-v2

Description:

Web tier AMI with corrected Internal ALB configuration

The AMI status became:

Available

PART 32 — WEB LAUNCH TEMPLATE

Created:

Web-Launch-Template

Configuration

AMI:

WebServer-AMI-v2

Key pair:

Don't include

Security Group:

Web-EC2-SG

Subnet:

Not selected

IAM Profile:

ThreeTierEC2Role-for-my-project

PART 33 — WEB AUTO SCALING GROUP

Created:

Web-ASG

Launch template:

Web-Launch-Template

VPC:

vpc-010be35b8756813cd

Subnets:

Public-Web-AZ1

Public-Web-AZ2

Target group:

Web-TG

Capacity

Desired: 2

Minimum: 2

Maximum: 2

The Web ASG automatically created Web instances in the two public subnets.

PART 34 — INTERNAL ALB CORRECTION

During testing, the first Internal-App-ALB was accidentally created as:

Internet-facing

This caused the internal API request to hang and eventually resulted in:

504 Gateway Timeout

The incorrect ALB was deleted and recreated with:

Scheme:

Internal

The corrected ALB became:

Internal-App-ALB

DNS:

internal-Internal-App-ALB-215633768.ap-south-1.elb.amazonaws.com

The new Nginx configuration was uploaded to WebServer-01.

PART 35 — INTERNAL ALB TEST

From WebServer-01:

curl http://internal-Internal-App-ALB-215633768.ap-south-1.elb.amazonaws.com/health

Output

"This is the health check"

Then:

curl http://internal-Internal-App-ALB-215633768.ap-south-1.elb.amazonaws.com/transaction

Output

{"result":[{"id":1,"amount":"400.00","description":"groceries"}]}

This verified:

WebServer-01

↓

Internal-App-ALB

↓

App Tier

↓

RDS MySQL

PART 36 — PUBLIC APPLICATION TEST

The Public ALB DNS name was opened in a browser:

http://public-web-alb-2107528530.ap-south-1.elb.amazonaws.com

The React application loaded successfully.

PART 37 — DATABASE DEMO TEST

Opened:

DB DEMO

The application displayed:

ID	AMOUNT	DESC
----	--------	------

1	400.00	groceries
---	--------	-----------

This demonstrated that the React frontend successfully retrieved data from the backend and database.

PART 38 — ADD TRANSACTION TEST

A new transaction was added through the React application.

Example

Amount:

500

Description:

test transaction

After clicking:

ADD

the application displayed:

ID	AMOUNT	DESC
----	--------	------

1	400.00	groceries
---	--------	-----------

2	500.00	test transaction
---	--------	------------------

This demonstrated the write operation:

React

↓

Public-Web-ALB

↓

Web Tier / Nginx

↓

Internal-App-ALB

↓

App Tier

↓

RDS MySQL

PART 39 — DATA PERSISTENCE TEST

The browser page was refreshed.

The newly added transaction remained visible.

Example:

2 500.00 test transaction

This confirmed that the transaction was stored in RDS rather than only existing in frontend memory.

PART 40 — FINAL TEST OUTPUTS

For the documentation, these are good sample outputs to include.

Application health

curl http://localhost:4000/health

Output

"This is the health check"

Application database API

curl http://localhost:4000/transaction

Output

```
{"result":[{"id":1,"amount":"400.00","description":"groceries"}]}
```

Internal ALB health

curl http://internal-Internal-App-ALB-215633768.ap-south-1.elb.amazonaws.com/health

Output

"This is the health check"

Internal ALB database API

curl http://internal-Internal-App-ALB-215633768.ap-south-1.elb.amazonaws.com/transaction

Output

```
{"result":[{"id":1,"amount":"400.00","description":"groceries"}]}
```

Nginx configuration test

```
sudo nginx -t
```

Output

```
syntax is ok
```

```
test is successful
```

Database verification

```
SELECT * FROM transactions;
```

Output

```
+----+-----+-----+
| id | amount | description |
+----+-----+-----+
| 1 | 400.00 | groceries  |
+----+-----+-----+
```

PART 41 — FINAL RESOURCES

The completed project contains:

VPC

|

|— Internet Gateway

|

|— Public-Web-AZ1

|— Public-Web-AZ2

|

|— Private-App-AZ1

|— Private-App-AZ2

|

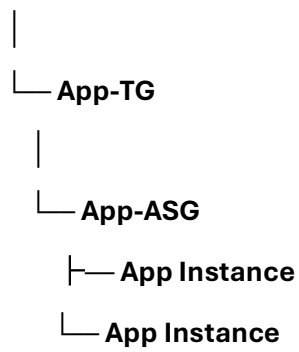
|— Private-DB-AZ1

|— Private-DB-AZ2

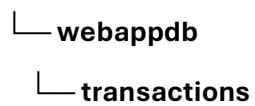
Public-Web-ALB



Internal-App-ALB



RDS MySQL



S3



IAM



